



Binary Trees

Escaping the Flat Array: Pointers, Recursion, and Hierarchical Data

2026 ACM Handout Series

100 Days Of Code

Abstract: An exhaustive guide to building, traversing, and manipulating Binary Trees in C. This handout transitions from linear data structures to dynamic, recursive, two-dimensional pointer architectures. Covers BST operations, four traversal strategies, memory management, debugging pitfalls, competitive programming tricks, and self-balancing fundamentals.

© 2026 ACM Handout Series
100 Days Of Code
All rights reserved.

Contents

Contents

1	The Problem	i
1.1	Why arrays fall short	i
1.2	Concrete example: the phone book problem	i
1.3	Trade-off comparison	i
1.4	Real-world analogy: the tournament bracket	i
1.5	When to use it	ii
2	Core Building Block: The Node	ii
2.1	Pointer structure diagram	iii
3	The Critical Concept: Recursive Structure	iii
4	Glossary of Terms	iv
5	Cross-Language Syntax	iv
6	Analogies	v
7	Static Diagram	v
7.1	Depth and height annotations	vi
8	Step-by-Step Walkthrough: Post-order Traversal	vii
8.1	Complexity Intuition	vii
9	Complexity Reference Table	viii
10	Golden Rules	viii
11	Algorithmic Patterns	viii
11.1	Pattern: Bottom-Up Recursion (Divide & Conquer)	viii
12	Full Code Examples	ix
12.1	Node Creation	ix
12.2	Inserting into a BST	ix
12.3	Traversal Methods	ix

12.4 Search in a BST	x
12.5 Level-Order Traversal (BFS)	x
12.6 Memory Management: Freeing the Tree	xi
12.7 Deletion in a BST	xi
12.8 Visualizing BST Deletion	xii
13 Bug Scenarios	xii
13.1 Bug 1: The Memory Leak (Pre-order Free)	xii
13.2 Bug 2: Dropping Pointers on Insert	xiii
13.3 Bug 3: The Leaf Check Trap	xiii
13.4 Bug 4: Dangling Pointers on Deletion	xiii
14 All Variants	xiv
15 Conceptual Deep-Dive	xiv
15.1 Why BSTs fail in production	xiv
16 Allocation Mechanics	xv
17 Problem Ladder	xvi
17.1 Hints	xvi
18 Complexity Table (Quick Reference)	xvi
19 Pattern Cheat Sheet	xvii
20 Platform Tips	xvii
21 Common Mistakes	xvii
22 More Bug Scenarios	xviii
22.1 Bug 5: Recursive Insert Without Return Value	xviii
22.2 Bug 6: Infinite Recursion	xviii
23 What You’ve Learned	xix
24 Forward-Looking Statement	xix

1 The Problem

1.1 Why arrays fall short

Arrays are the simplest data structure, but they impose a rigidity that breaks down in real-world scenarios:

- **Inserting in the middle** requires shifting every subsequent element — $\mathcal{O}(n)$ time.
- **Searching** through an unsorted array is $\mathcal{O}(n)$; even a sorted array requires $\mathcal{O}(n)$ for inserts.
- **Hierarchical data** (file systems, HTML DOM, parse trees, organizational charts) simply cannot be represented naturally in a flat, linear structure.

A **Binary Tree** solves all three: it supports $\mathcal{O}(\log n)$ search and insert (when balanced), and directly models hierarchical relationships through pointer-connected nodes.

1.2 Concrete example: the phone book problem

You maintain a sorted array of 10,000 names. Adding a new name means finding the insertion point and shifting up to 10,000 entries. A Binary Search Tree inserts the new name in $\mathcal{O}(\log n)$ pointer operations — no shifting, no copying.

1.3 Trade-off comparison

Feature	Sorted Array	Linked List	BST (Balanced)
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Insert	$\mathcal{O}(n)$	$\mathcal{O}(1)$ (at head)	$\mathcal{O}(\log n)$
Delete	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Memory	Contiguous	Fragmented	Fragmented
Cache-friendly	Yes	No	No
Hierarchical data	No	No	Yes
<i>$\mathcal{O}(\log n)$ for BST assumes a balanced tree; worst case is $\mathcal{O}(n)$ (skewed).</i>			

Table 1: Comparison of linear data structures vs. Binary Search Tree

1.4 Real-world analogy: the tournament bracket

Imagine a knockout tennis tournament with 64 players. The bracket works like this:

- Player A plays Player B; the winner advances. That match is a **leaf-level decision**.
- The winner of (A vs B) plays the winner of (C vs D). That's one level up.
- This repeated halving continues until one champion remains — the **root**.

A Binary Search Tree works the same way: to find a player in the bracket, you start at the final (root) and ask “did this player come from the left half or the right half?” Each question eliminates half the remaining players. With 64 players, you reach any specific player in just 6 decisions ($\log_2 64 = 6$). With 1,000,000, it takes 20.

This is the core intuition for $\mathcal{O}(\log n)$ search: each comparison halves the search space, just like each match in a bracket halves the remaining contenders.

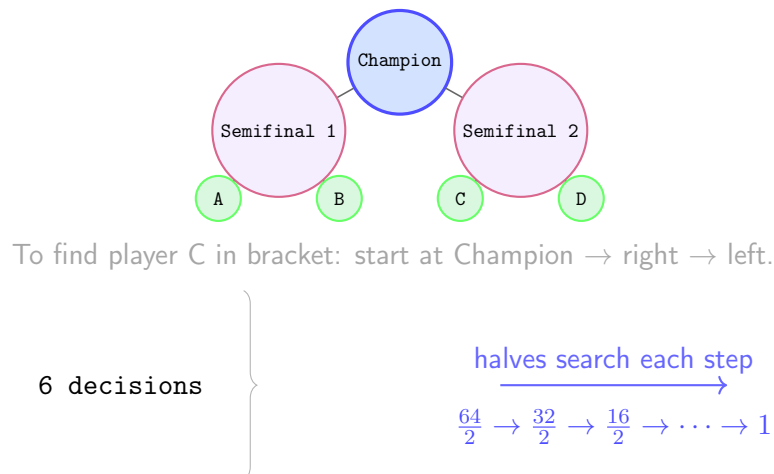


Figure 1: Tournament bracket as a binary tree: each match halves the contenders, just like each comparison in a BST halves the search space

1.5 When to use it

Use a Binary Tree when:

- You need fast search, insert, and delete — all $\mathcal{O}(\log n)$.
- Data is inherently hierarchical (file paths, taxonomies, expression trees).
- You need sorted data on demand (In-order traversal of a BST).

Avoid a Binary Tree when:

- A simple array suffices (fixed-size, static data).
- You need constant-time random access (use an array or hash table).
- Your data is small ($n < 50$), where linear structures with better cache locality outperform the tree’s pointer-chasing overhead.

2 Core Building Block: The Node

A Binary Tree is built from individual **nodes**. Every node contains exactly three fields:

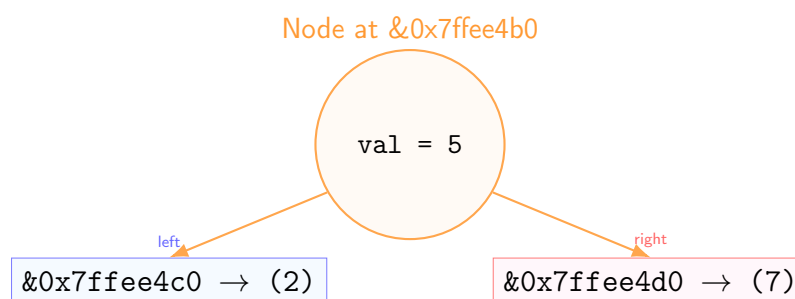
- `int val` — the data payload

- `TreeNode* left` — pointer to the left child
- `TreeNode* right` — pointer to the right child

Listing 1: Binary tree node definition

```
1 typedef struct TreeNode {  
2     int val;  
3     struct TreeNode* left;  
4     struct TreeNode* right;  
5 } TreeNode;
```

2.1 Pointer structure diagram

Figure 2: Anatomy of a `TreeNode` struct in memory

3 The Critical Concept: Recursive Structure

A Binary Tree is **recursively defined**: a tree is either:

- **Empty** (NULL pointer) — the base case, or
- **A node** with a value, a left subtree, and a right subtree.

This recursive definition leads directly to recursive algorithms:

Listing 2: The universal pattern of recursive tree functions

```
1 void traverse(TreeNode* root) {  
2     // 1. BASE CASE: empty tree  
3     if (root == NULL) return;  
4  
5     // 2. Do something with root      <-- Pre-order position  
6  
7     // 3. Recurse on children  
8     traverse(root->left);  
9  
10    // 4. Do something with root      <-- In-order position  
11  
12    traverse(root->right);
```

```

13
14     // 5. Do something with root      <-- Post-order position
15 }

```

Pro Tip

The **first line** of any recursive tree function should be a NULL check. This is non-negotiable. Forgetting it is the #1 source of tree-related segfaults.

4 Glossary of Terms

Root The topmost node of the tree (no parent).

Child A node directly connected to another node when moving away from the root.

Parent A node directly connected to another node when moving toward the root.

Leaf A node with no children.

Internal Node A node with at least one child.

Depth Number of edges from root to a given node.

Height Number of edges from a node to its deepest leaf. The tree's overall height is the height of the root.

Subtree A tree consisting of a node and all its descendants.

Level All nodes at a given depth. A tree with h rows (root to deepest leaf) has levels 0 through $h - 1$. When someone says “the tree has L levels”, they mean $L = h$, the total row count — not the level index.

5 Cross-Language Syntax

Language	Node Definition	Allocation	Child Access
C	<code>struct TreeNode {...}</code>	<code>malloc(...)</code>	<code>root->left</code>
C++	<code>class Node {...}</code>	<code>new Node(...)</code>	<code>root->left</code>
Python	<code>class TreeNode:</code>	<code>TreeNode(val)</code>	<code>root.left</code>
Java	<code>class Node {...}</code>	<code>new Node(val)</code>	<code>root.left</code>
<i>In all languages, the core three-field structure (value, left, right) is identical.</i>			

Table 2: Binary tree node definitions across languages

6 Analogies

- **The Corporate Org Chart:** The CEO is the root. VPs are internal nodes. Individual contributors are leaves. Information flows down (commands) and flows up (reports).
- **The Family Tree (paternal side):** The great-grandfather is the root (single source). His child (your grandparent) is an internal node. Your parent is another internal node, and you (or your siblings) are the leaves. Following a parent pointer is like tracing ancestry backward through generations.
- **The Tournament Bracket:** Each match combines two winners into one. The final winner is the root, the initial teams are the leaves.

7 Static Diagram

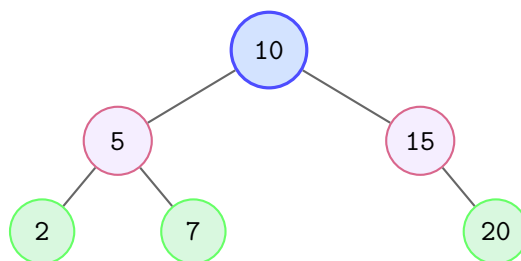


Figure 3: A standard binary tree showing root (10), internal nodes (5, 15), and leaf nodes (2, 7, 20)

7.1 Depth and height annotations

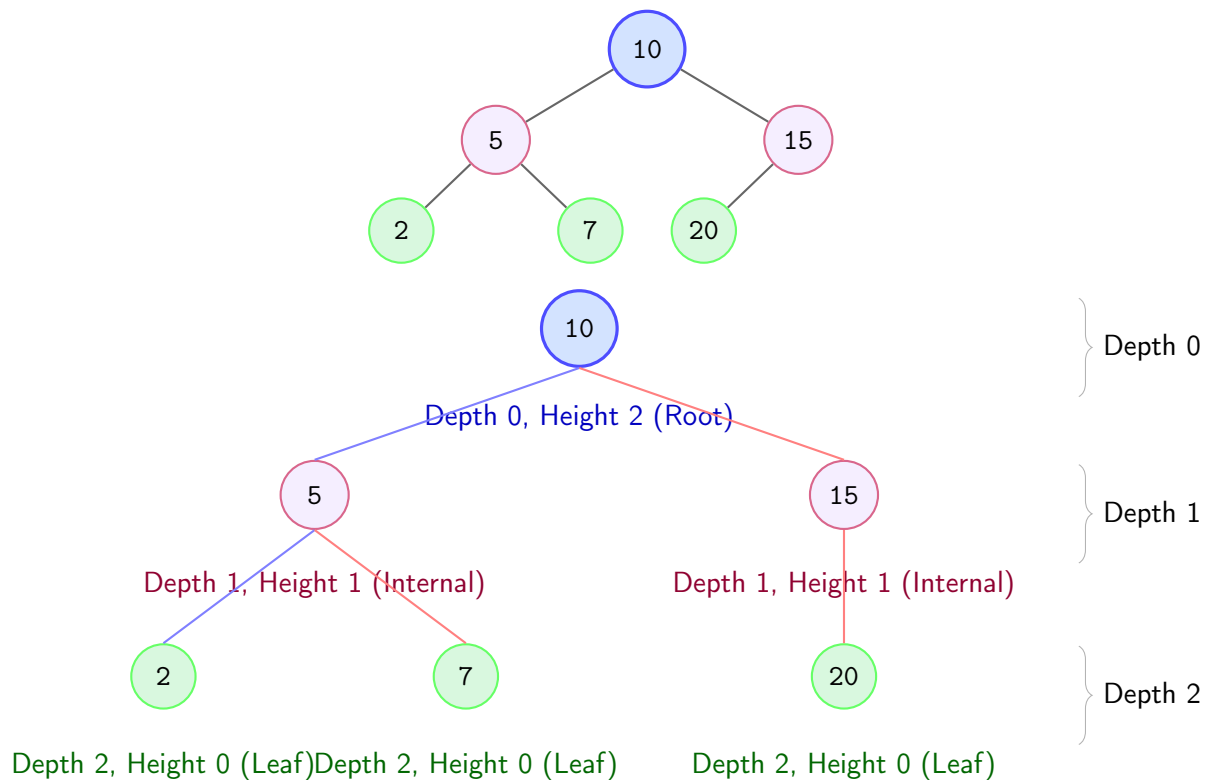


Figure 4: Tree annotated with depth and height measurements

Depth & Height — Definitions and Formulas

Depth of a node = the number of *edges* on the path from the root down to that node.

- Base case: $\text{depth}(\text{root}) = 0$
- Recursive: $\text{depth}(v) = \text{depth}(\text{parent}(v)) + 1$

Height of a node = the number of *edges* on the longest downward path from that node to a leaf.

- Base case: $\text{height}(\text{leaf}) = 0$ (equivalently, $\text{height}(\text{NULL}) = -1$; the -1 ensures leaves compute to $1 + \max(-1, -1) = 0$)
- Recursive: $\text{height}(v) = 1 + \max(\text{height}(v.\text{left}), \text{height}(v.\text{right}))$

Height of the tree = $\text{height}(\text{root})$. For this example tree: $\text{height} = 2$.

Levels vs. Height: A tree of height h has $h + 1$ levels (rows) numbered 0 through h . For this tree ($h = 2$): levels are 0, 1, 2 — three rows total. If a problem says “the tree has 6 levels”, it means $h = 5$ (levels 0–5). The last level (index h) contains the leaf nodes.

8 Step-by-Step Walkthrough: Post-order Traversal

Traverse the tree in **Left-Right-Root** order.

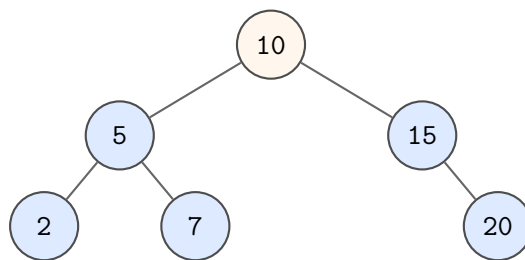


Figure 5: Starting at root (10), we go left to node (5) before printing anything

1. Start at **10**. Go Left to **5**.
2. At **5**. Go Left to **2**.
3. At **2**. Left is NULL, Right is NULL. **Print 2**. Return to **5**.
4. At **5**. Go Right to **7**.
5. At **7**. Left is NULL, Right is NULL. **Print 7**. Return to **5**.
6. At **5**. Left and Right done. **Print 5**. Return to **10**.
7. At **10**. Go Right to **15**.
8. At **15**. Left is NULL. Go Right to **20**.
9. At **20**. Left is NULL, Right is NULL. **Print 20**. Return to **15**.
10. At **15**. Left and Right done. **Print 15**. Return to **10**.
11. At **10**. Left and Right done. **Print 10**.

Output: 2, 7, 5, 20, 15, 10

8.1 Complexity Intuition

- **Why is balanced BST search $\mathcal{O}(\log n)$?** With 1,000,000 nodes, the root's left/right choice instantly eliminates 500,000. Each subsequent decision halves the remaining search space. It takes only ~ 20 decisions ($\log_2 1,000,000$) to reach any node.
- **Why do skewed trees fail?** Inserting sorted data (1, 2, 3, 4) into a BST never branches left — it becomes a linked list, degrading search to $\mathcal{O}(n)$.

9 Complexity Reference Table

Operation (BST)	Best (Balanced)	Worst (Skewed)	Call Stack
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n) — \mathcal{O}(n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n) — \mathcal{O}(n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n) — \mathcal{O}(n)$
Traversal (DFS)	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n) — \mathcal{O}(n)$

Table 3: Time and space complexity of BST operations

10 Golden Rules

- **“Base case first.”** The first line of any recursive tree function must be `if (root == NULL) return ...;`
- **“Pre = Copy, Post = Delete, In = Sort.”** Use Pre-order to clone a tree, Post-order to free memory, and In-order to print a BST in ascending order.
- **“Updates flow upward.”** When inserting or deleting, return the modified pointer up the call stack: `root->left = insert(root->left, val);`

11 Algorithmic Patterns

11.1 Pattern: Bottom-Up Recursion (Divide & Conquer)

Problem: Find the height of a binary tree.

Insight: I don’t know my height. But if I ask my left child and right child their heights, my height is `max(left, right) + 1`.

Listing 3: Recursive tree height computation

```

1  int max(int a, int b) { return (a > b) ? a : b; }
2
3  int treeHeight(TreeNode* root) {
4      if (root == NULL) return 0;           // Base case
5
6      int leftHeight = treeHeight(root->left); // Ask left
7      int rightHeight = treeHeight(root->right); // Ask right
8
9      return max(leftHeight, rightHeight) + 1; // Combine
10 }
```

Complexity: Time $\mathcal{O}(n)$, Space $\mathcal{O}(h)$ where h is tree height.

12 Full Code Examples

12.1 Node Creation

Listing 4: Helper function to create a new tree node

```
1  TreeNode* createNode(int data) {  
2      TreeNode* newNode = (TreeNode*)malloc(sizeof(TreeNode));  
3      if (newNode == NULL) {  
4          printf("Memory allocation failed!\n");  
5          exit(1);  
6      }  
7      newNode->val = data;  
8      newNode->left = NULL;  
9      newNode->right = NULL;  
10     return newNode;  
11 }
```

12.2 Inserting into a BST

Listing 5: Recursive insertion into a Binary Search Tree

```
1  TreeNode* insertBST(TreeNode* root, int data) {  
2      // 1. Base case: found insertion spot  
3      if (root == NULL) {  
4          return createNode(data);  
5      }  
6  
7      // 2. Recursive step: go left or right  
8      if (data < root->val) {  
9          root->left = insertBST(root->left, data);  
10     } else if (data > root->val) {  
11         root->right = insertBST(root->right, data);  
12     }  
13     // If equal, do nothing (no duplicates)  
14  
15     // 3. Return unchanged root pointer upward  
16     return root;  
17 }
```

12.3 Traversal Methods

Listing 6: DFS: Pre-order, In-order, and Post-order traversals

```
1  // Pre-order: Root → Left → Right  
2  void preorder(TreeNode* root) {  
3      if (root == NULL) return;  
4      printf("%d ", root->val); // Root first (for copying)
```

```

5     preorder(root→left);
6     preorder(root→right);
7 }
8
9 // In-order: Left →Root →Right
10 void inorder(TreeNode* root) {
11     if (root == NULL) return;
12     inorder(root→left);
13     printf("%d ", root→val); // Root in middle (sorted output)
14     inorder(root→right);
15 }
16
17 // Post-order: Left →Right →Root
18 void postorder(TreeNode* root) {
19     if (root == NULL) return;
20     postorder(root→left);
21     postorder(root→right);
22     printf("%d ", root→val); // Root last (for deletion)
23 }

```

12.4 Search in a BST

Listing 7: BST search returning pointer to node or NULL

```

1  TreeNode* searchBST(TreeNode* root, int target) {
2      // Base case: not found or empty tree
3      if (root == NULL || root→val == target) {
4          return root;
5      }
6
7      // Recurse based on comparison
8      if (target < root→val) {
9          return searchBST(root→left, target);
10     } else {
11         return searchBST(root→right, target);
12     }
13 }

```

12.5 Level-Order Traversal (BFS)

Listing 8: BFS using an explicit queue

```

1  #define MAX_QUEUE 100000
2
3  void levelOrder(TreeNode* root) {
4      if (root == NULL) return;
5
6      TreeNode* queue[MAX_QUEUE];
7      int front = 0, rear = 0;

```

```

8
9     queue[rear++] = root; // Enqueue root
10
11     while (front < rear) {
12         TreeNode* current = queue[front++]; // Dequeue
13         printf("%d ", current->val);
14
15         if (current->left) queue[rear++] = current->left;
16         if (current->right) queue[rear++] = current->right;
17     }
18 }

```

12.6 Memory Management: Freeing the Tree

Listing 9: Post-order free (must use post-order!)

```

1 // MUST use Post-order. If you free root first, you lose access
2 // to left and right pointers!
3 void freeTree(TreeNode* root) {
4     if (root == NULL) return;
5
6     freeTree(root->left); // Free left subtree first
7     freeTree(root->right); // Then right subtree
8     free(root);          // Then the node itself
9 }

```

Warning

Using Pre-order to free a tree (`free(root)` before the recursive calls) causes a segfault — you access `root->left` after freeing `root`, which is undefined behavior.

12.7 Deletion in a BST

Listing 10: BST deletion handling all three cases

```

1 // Find the minimum value node in a subtree
2 // Assumes node ≠ NULL (guaranteed by call-site contract)
3 TreeNode* minValueNode(TreeNode* node) {
4     TreeNode* current = node;
5     while (current && current->left ≠ NULL)
6         current = current->left;
7     return current;
8 }
9
10 TreeNode* deleteBST(TreeNode* root, int data) {
11     if (root == NULL) return root;
12
13     // Find the node to delete
14     if (data < root->val)

```

```

15     root->left = deleteBST(root->left, data);
16 else if (data > root->val)
17     root->right = deleteBST(root->right, data);
18 else {
19     // Case 1: Leaf or one child
20     if (root->left == NULL) {
21         TreeNode* temp = root->right;
22         free(root);
23         return temp;
24     } else if (root->right == NULL) {
25         TreeNode* temp = root->left;
26         free(root);
27         return temp;
28     }
29
30     // Case 2: Two children
31     TreeNode* temp = minValueNode(root->right);
32     root->val = temp->val; // Copy inorder successor's value
33     root->right = deleteBST(root->right, temp->val);
34 }
35 return root;
36 }

```

12.8 Visualizing BST Deletion

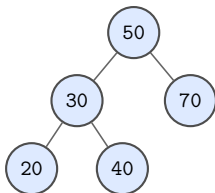


Figure 6: Before

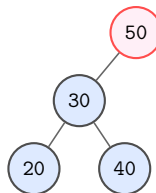


Figure 7: Delete 70 (leaf)

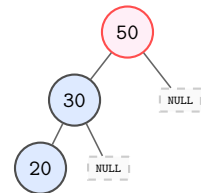


Figure 8: Delete 40 (one child)

Figure 9: Visualizing BST deletion cases: leaf (simple free), one child (bypass), and two children (inorder successor)

13 Bug Scenarios

13.1 Bug 1: The Memory Leak (Pre-order Free)

Listing 11: Wrong: Pre-order free causes segfault

```

1 void badFree(TreeNode* root) {
2     if (root == NULL) return;
3     free(root);           // Root freed first!
4     badFree(root->left);   // SEGFAULT! root->left accesses freed memory
5     badFree(root->right);
6 }

```

What breaks: Accessing `root->left` reads memory returned to the OS. Undefined behavior.

Correct code: Call `free(root)` after both recursive calls (Post-order).

Key lesson: A parent cannot die until its children are safely disposed of.

13.2 Bug 2: Dropping Pointers on Insert

Listing 12: Wrong: Pointer passed by value is lost

```
1 void insert(TreeNode* root, int val) {  
2     if (root == NULL) {  
3         root = createNode(val); // Local pointer only!  
4     }  
5     // ...  
6 }
```

What breaks: C passes pointers by value. The local `root` points to the new node, but the caller's pointer remains `NULL`. The call `insert(root, 5)` leaves the caller's `root` still `NULL` — the node floats off, a memory leak.

Correct code: Return `TreeNode*` and reassign (`root->left = insert(...)`), or use a double pointer (`TreeNode** root`).

13.3 Bug 3: The Leaf Check Trap

Listing 13: Wrong: Assuming root is never NULL

```
1 int sum(TreeNode* root) {  
2     if (root->left == NULL && root->right == NULL)  
3         return root->val; // Crashes if root is NULL!  
4     return root->val + sum(root->left) + sum(root->right);  
5 }
```

What breaks: Passing an empty tree (`root == NULL`) dereferences `NULL`. Also fails for nodes with only one child.

Correct code: `if (root == NULL) return 0;`

13.4 Bug 4: Dangling Pointers on Deletion

Bug: When deleting a leaf node, you call `free(targetNode)`, but the parent still holds the address in its `left` or `right` field.

Fix: The deletion function must set the parent's pointer to `NULL` after freeing.

14 All Variants

- **Full / Proper Binary Tree:** Every node has exactly 0 or 2 children. No nodes with just one child.
- **Complete Binary Tree:** Every level is filled completely, except possibly the last, which is filled left to right. Crucial for Heaps.
- **Perfect Binary Tree:** All internal nodes have two children, and all leaves are at the same depth.
- **Binary Search Tree (BST):** Left subtree values $<$ parent $<$ right subtree values.
- **Skewed Tree:** Every node has one child — effectively a linked list. Worst-case performance.

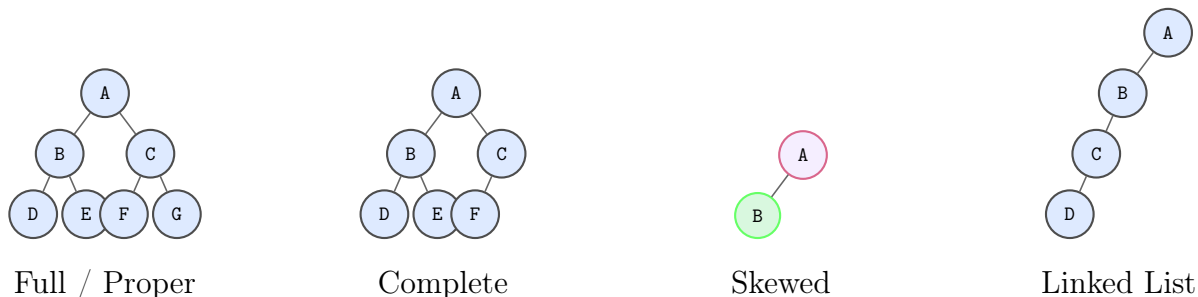


Figure 10: Four binary tree variants: Full/Proper, Complete, Skewed, and Linked List (pathological)

15 Conceptual Deep-Dive

15.1 Why BSTs fail in production

Inserting sequential data (10, 20, 30, 40) into a standard BST degrades it into a linked list. Search collapses from $\mathcal{O}(\log n)$ to $\mathcal{O}(n)$.

Deep Dive

Self-Balancing Trees (AVL / Red-Black) maintain extra metadata in the node struct — a balance factor or a color bit. During insertion, when the tree detects one side is getting too deep, it performs **tree rotations** to pivot back to balance without breaking BST ordering. The key insight: a rotation is just pointer reassignment — three pointers change, and the tree rebalances in $\mathcal{O}(1)$ time.

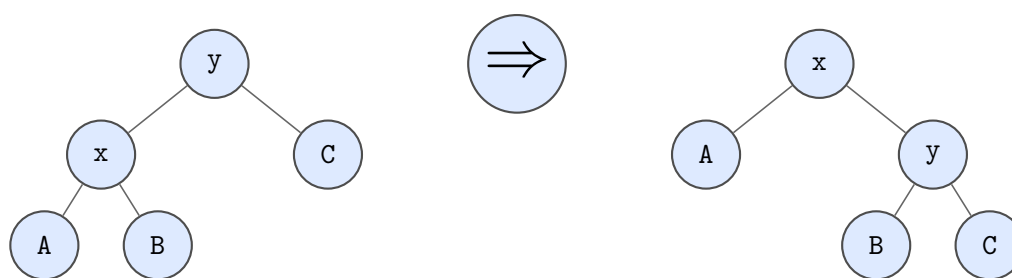


Figure 11: A right rotation on node y . Subtrees A, B, C preserve BST ordering throughout.

16 Allocation Mechanics

Trees stress two different areas of memory simultaneously:

1. **The Heap:** Every `createNode()` calls `malloc`, grabbing ~ 24 bytes (int + 2 pointers) at a scattered heap location. Trees are cache-unfriendly — nodes at random RAM addresses cause frequent cache misses on traversal.
2. **The Call Stack:** Every recursive step pushes a stack frame. A balanced tree of 1M nodes recurses 20 deep ($\log_2 n$) — safe. A skewed tree of 100k nodes recurses 100k deep — **stack overflow**.

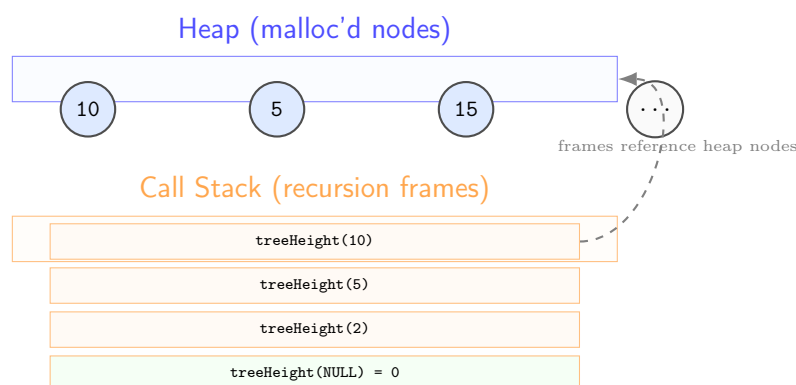


Figure 12: Heap and call stack interaction during recursive tree traversal

17 Problem Ladder

Practice Problems

1. Level 1 (Foundations):

- Find the maximum depth of a binary tree.
- Sum all nodes in a tree.
- Count the number of leaf nodes.

2. Level 2 (Application):

- Invert a Binary Tree (swap all left and right pointers).
- Find the Lowest Common Ancestor (LCA) of two nodes.
- Check if two trees are identical.

3. Level 3 (Advanced):

- Serialize and Deserialize a Binary Tree.
- Validate if a tree is a valid BST.
- Diameter of a Binary Tree.

17.1 Hints

- **Max Depth:** Think bottom-up. $1 + \max(\text{leftDepth}, \text{rightDepth})$.
- **Invert:** Think top-down. Swap left and right, then command them to invert themselves.
- **Validate BST:** Passing just the root is not enough. Pass `min` and `max` constraints down the recursion.
- **LCA:** If one node is in the left subtree and the other is in the right, the current node is the LCA.

18 Complexity Table (Quick Reference)

Operation	Best/Avg (Balanced)	Worst (Skewed)
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$

Table 4: Big-O complexity of BST core operations

19 Pattern Cheat Sheet

- Need to process children before parent? → Post-order (deletion, height calculation).
- Need to process parent before children? → Pre-order (copying, serialization).
- Need sorted data from a BST? → In-order.
- Need to search level by level? → BFS / Level-Order (requires explicit queue).
- Need to check property on all root-to-leaf paths? → Pre-order with parameters passed down.
- Need aggregated result from entire tree? → Post-order with return values passed up.

20 Platform Tips

- **1D Array Representation:** In competitive programming, `malloc` per node is too slow. Flatten the tree into an array: root at index i , left child at $2i$, right child at $2i + 1$. Cache-friendly and infinitely faster. Heavily used in Segment Trees and Heaps.
- **Global Array Pools:** If pointer trees are required, allocate a massive array upfront:

Listing 14: Custom allocator for competitive programming (global scope)

```
1 // Global pool to avoid malloc overhead
2 TreeNode pool[100000];
3 int pool_idx = 0;
4
5 TreeNode* newNode(int val) {
6     pool[pool_idx].val = val;
7     pool[pool_idx].left = NULL;
8     pool[pool_idx].right = NULL;
9     return &pool[pool_idx++];
10 }
```

- **Iterative DFS:** Avoid recursion depth issues on skewed trees. Use an explicit stack.

21 Common Mistakes

- **Global Variable Accumulators:** Using `int count = 0;` globally to count nodes fails across multiple test cases. Always pass state via parameters or return values.
- **Assuming Distinct Values:** Failing to handle duplicate values in BST insertions. Deci-

sion needed: left, right, or ignore?

- **Forgetting the NULL Check:** The first operation on any tree pointer must verify it's not NULL.

22 More Bug Scenarios

22.1 Bug 5: Recursive Insert Without Return Value

Listing 15: Wrong: Inserting without reassigning the return value

```
1 void insertBST(TreeNode* root, int data) {
2     if (root == NULL) {
3         root = createNode(data); // Wrong! root is local
4         return;
5     }
6     if (data < root->val)
7         insertBST(root->left, data); // root->left still NULL!
8     else if (data > root->val)
9         insertBST(root->right, data);
10 }
```

What breaks: Both the initial NULL case and the recursive calls modify only local copies of the pointer. The tree structure never changes.

Correct approach: Always return the (potentially modified) node pointer:

Listing 16: Correct: Returning modified pointer

```
1 TreeNode* insertBST(TreeNode* root, int data) {
2     if (root == NULL) return createNode(data);
3     if (data < root->val)
4         root->left = insertBST(root->left, data);
5     else if (data > root->val)
6         root->right = insertBST(root->right, data);
7     return root;
8 }
```

22.2 Bug 6: Infinite Recursion

Listing 17: Wrong: Missing base case causes stack overflow

```
1 void printTree(TreeNode* root) {
2     printf("%d ", root->val); // No NULLcheck!
3     printTree(root->left);
4     printTree(root->right);
5 }
```

What breaks: When reaching a NULL child, the function dereferences NULL before the printf — segfault. If somehow surviving that, `root->left` and `root->right` on a NULL pointer is undefined behavior.

Fix: `if (root == NULL) return;` as the first line.

Debugging Exercise

Debugging Exercise: Run the following and observe the output:

Listing 18: Debugging: Tracing recursive tree traversal

```
1 void debugTraverse(TreeNode* root, int depth) {  
2     if (root == NULL) {  
3         printf("%*s[NULL]\n", depth * 4, "");  
4         return;  
5     }  
6     printf("%*sNode(%d) at depth %d\n",  
7         depth * 4, "", root->val, depth);  
8     debugTraverse(root->left, depth + 1);  
9     debugTraverse(root->right, depth + 1);  
10 }
```

The `%*s` format specifier with `depth * 4` indents deeper nodes, visually revealing the recursion structure. Run this on a skewed tree to see how deep the recursion goes.

23 What You’ve Learned

You have moved beyond flat, rigid arrays into recursive pointer logic. You understand how to:

- Dynamically allocate and link tree nodes using heap memory.
- Leverage the Call Stack for recursive DFS traversals.
- Exploit geometric halving of BSTs for $\mathcal{O}(\log n)$ search.
- Recognize the fatal flaws: unbalanced trees $\rightarrow$ linked lists, recursive depth $\rightarrow$ stack overflow.
- Manage memory safely with post-order deletion.

24 Forward-Looking Statement

Binary Trees are powerful, but leaving their balance to chance is dangerous. Furthermore, trees enforce a strict parent-child hierarchy. What if data naturally forms loops — like social networks or city roads?

The next step: **Graph Theory** — adjacency lists vs. matrices, cycle detection, and the full

generality of BFS/DFS without tree constraints.